

Individual Contribution Reflection

Tom Tian

COMP4347 / COMP5347 Assignment 2: MERN Quiz Game

Subsystem D: integration, validation, robustness, shared shell, documentation, and tests

1 ROLE, OWNERSHIP, AND RUBRIC COVERAGE

In our four-person team, my contribution was the integration and validation role required for a group of four. I was not the primary owner of authentication, quiz business logic, leaderboard data, or admin CRUD. My work was to make those independently developed parts behave consistently as one MERN application, and to leave evidence that the final system matched the assignment specification.

Rubric area	How my work contributed
Backend Architecture & APIs	Assembled shared Express app behaviour in <code>backend/src/app.js</code> , response envelopes, central error handling, route documentation, and Jest/Supertest coverage.
Security & Validation	Added and checked rate-limit responses, protected-route failure responses, envelope consistency, and Swagger status-code alignment for 401, 403, and 429.
Frontend UI & UX	Scaffolded the initial shared React shell, API client, route wiring, and persisted theme that teammates' feature pages were then built into; I did not own the individual feature pages.
Documentation & Process	Produced README/API/Postman/architecture evidence so the marker can verify the system without guessing which branch or endpoint is authoritative.

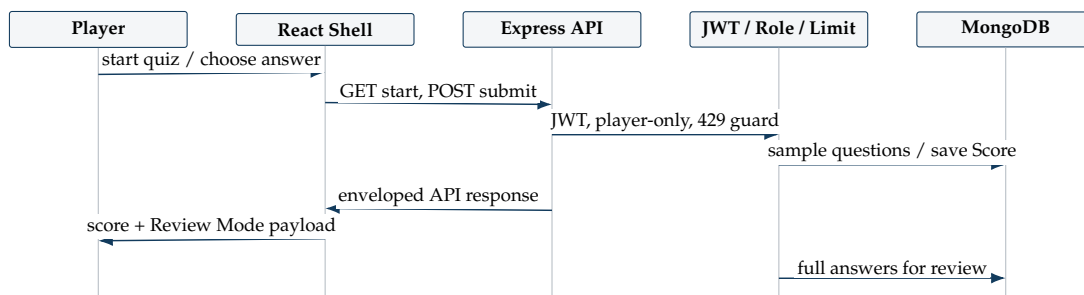
2 MAJOR TECHNICAL CHALLENGE

The hardest issue I handled was not a visual bug; it was a contract bug. The Review Mode variation depends on the same answer order being used in four places: the shuffled options returned by `/api/quiz/start`, the visible index submitted by the frontend, the backend scoring logic, and the saved payload later shown in Review Mode. A test still assumed the raw seed answer index, which was unsafe after option shuffling. That kind of bug is easy to miss because the UI can still look plausible while explaining the wrong answer.

I fixed the backend API test in `backend/src/tests/api.test.js` so it follows the real user path: start quiz, locate the correct visible option in the shuffled list, submit that visible index, and then assert that the saved review payload matches the same ordering. Commit [bba8b06](#) is the direct fix. I then tightened the surrounding API documentation in [4a8fe22](#) and [d0d07f6](#), because undocumented 403/429 paths would make the integration contract harder to review.

start quiz -> read shuffled options -> submit selected visible index -> assert saved Review Mode payload

3 SUBSYSTEM UML SEQUENCE DIAGRAM



4 REFLECTION ON THE APPROVED VARIATION

Our approved variation was Review Mode after completion. My design view was that Review Mode should not change the core scoring rule: the player still earns +1 per correct answer and no extra scoring scheme is introduced. The extra responsibility is data integrity after the attempt. Because answers are shuffled, the system must preserve enough information to reconstruct what the player saw, selected, and got right or wrong. This is why full answer-list persistence is necessary for our group even though staff clarified that simpler non-review projects may persist only the final score.

In practice, this gave me three integration checks to protect: the start response should not leak `correctAnswer`; submit should score the visible selected index in the shuffled list; and history/review should explain the saved attempt rather than reconstructing a different answer order later. Those checks shaped the backend API test fix and the final Swagger response audit.

5 COMMIT HISTORY ANALYSIS

Hash	Focus	Contribution evidence
b176192	workspace scaffold	Created the initial workspace so backend, frontend, scripts, and checks could be developed together.
b6d0ffe	backend app foundation	Established the Express foundation later used by routes, middleware, and integration tests.
57ebccb	shared rate limiters	Added login and quiz-submit throttling with response-envelope compatible failure handling.
d7c20c6	frontend providers	Integrated shared providers for routing, auth-aware API use, and theme behaviour.
51c7e39	shared app shell	Connected public, player, and admin flows inside one React application shell.
8501ac2	Swagger documentation	Added self-hosted API documentation for marker and teammate contract review.
7acd98d	envelope/error tests	Verified success and failure envelope behaviour instead of relying on manual inspection.
03a5d8e	architecture docs	Documented integration decisions and architecture rationale for submission review.
2bda2bb	security/API evidence	Added docs/security-validation.md, docs/manual-test-checklist.md, and docs/postman-collection.json so JWT, rate-limit, and envelope behaviour were reviewable.
6686eae	D seed/docs/tests	Expanded seed support, Postman evidence, and D-owned regression coverage.
972fc77	final QA alignment	Reconciled tests and documentation with the delivered API surface.
160ac65	leaderboard QA note	Added a README note flagging that leaderboard auth posture was under final review, so the QA risk was not left implicit.
5d4139b	quiz readiness note	Reworded the quiz-session line in docs/delivery-readiness.md so the readiness doc no longer disagreed with the delivered Review Mode contract.
bba8b06	shuffled answer tests	Fixed the test contract for shuffled visible options and saved review payloads.
d0d07f6	quiz error responses	Completed Swagger alignment for player-only and rate-limited quiz routes.

6 FINAL VERIFICATION AND WHAT I WOULD DEFEND IN MARKING

The common theme in my commits is integration ownership: making separately built features testable together. The part I would defend most strongly is the Review Mode contract. If `/api/quiz/start`, `/api/quiz/submit`, saved scores, and review rendering disagree, the project may appear to work while giving incorrect feedback. Before tagging the submission branch, I ran `npm test --prefix backend`, `npm run build --prefix frontend`, `python3 -m json.tool docs/postman-collection.json`, and `git diff --check; root, backend, and frontend npm audit` all reported zero vulnerabilities at the time of submission.

I also checked the failure paths that usually cause marking problems in group projects: unauthenticated API access should fail as 401; admins should not be able to take player quizzes and should receive 403; repeated quiz submissions should be rate-limited with 429; and every documented failure should still use the shared response envelope. These are small details, but they are exactly where separately implemented subsystems tend to drift before submission.

The single commit I would defend most strongly is `bba8b06` – the Review Mode test contract. That is the failure mode that would have let the application appear to work while explaining a player's answer incorrectly.

7 OWNERSHIP BOUNDARY AND LESSONS

The main lesson from this subsystem was that integration work is not the same as taking over other members' features. I could have resolved some cross-owner issues by editing quiz or admin business logic directly, but that would have crossed the ownership boundary the group-of-four integration role defines. Instead, I focused on the shared contracts around those features: tests, route documentation, response formats, and delivery-readiness notes. That kept the work aligned with the group-of-four requirement while still reducing real submission risk.

For future group projects, I would start the route-documentation cross-check earlier. The late Swagger drift around player-only quiz routes showed that a route can be technically correct while its documented contract is stale. Catching those mismatches earlier would reduce final-week pressure and make peer review easier for the whole team.